

Chapitre 5 Structures itératives

Les ordinateurs ont été développés à l'origine pour exécuter de manière répétitive et de nombreuses fois des calculs qu'il aurait été impossible de réaliser à la main. Les structures itératives, au nombre de trois en C++ (boucle `for`, boucle `while`, boucle `do...while`), exécutent des instructions de manière répétée (exemple : réaliser 100 mesures de température).

Boucle `while` `(expression) { ... }`

La boucle `while` `(expression) { instructions ; }` exécute les instructions qui la composent tant que l'expression est vraie.

Syntaxe

```
while (expression)
    instruction;
```

ou

```
while (expression)
{
    instruction1;
    instruction2;
    ...
}
```

Remarques

- L'expression doit être de type entier (booléen), c'est-à-dire valoir la valeur 0 (« faux ») ou toute valeur différente de 0 (considérée alors comme « vrai »).
- L'expression est évaluée **AVANT** l'exécution des instructions.
- Les instructions sont exécutées **TANT QUE** l'expression est vraie.
- L'expression est évaluée à chaque fois que toutes les instructions de la boucle sont exécutées.
- Lorsque l'expression est ou devient fausse, les instructions de la boucle `while` `(expression) { instructions ; }` ne sont plus exécutées et le programme continue son exécution.
- Pour éviter que la boucle, et donc le programme, ne s'exécute sans fin, le programmeur doit faire évoluer l'expression à chaque exécution de la boucle.

Exemple

Afficher des entiers consécutifs en utilisant une boucle `while` `(expression) { instructions ; }`

```
int main(void)
{
    cout << "=== Afficher des entiers cons\202cutifs ===" << endl;

    int nPremier = 0;
    cout << "Entrez le premier entier : "; cin >> nPremier;
```

```

int nDernier = 0;
cout << "Entrez le dernier entier : "; cin >> nDernier;

int nN = nPremier;
cout << "\tListe des entiers : ";

while (nN <= nDernier)
{
    cout << nN << " ";
    ++nN;
}

```

Boucle `do {...} while (expression)`

Comme la boucle `while (expression) { instructions ; }`, la boucle `do { instructions ; } while (expression);` exécute les instructions qui la composent tant que l'expression est vraie.

Syntaxe

```

do
    instruction;
while (expression);

ou

do
{
    instruction1;
    instruction2;
    ...
} while (expression);

```

Remarques

- L'expression doit être de type entier (booléen), c'est-à-dire valoir la valeur 0 (« faux ») ou toute valeur différente de 0 (considérée alors comme « vrai »).
- L'expression est évaluée **APRÈS** l'exécution des instructions. Les instructions de la boucle `do { instructions; } while (expression);` sont donc exécutées **AU MOINS** une fois.
- Les instructions sont exécutées **TANT QUE** l'expression est vraie.
- L'expression est évaluée à chaque fois que toutes les instructions de la boucle sont exécutées.
- Lorsque l'expression est ou devient fausse, les instructions de la boucle `do { instructions; } while (expression);` ne sont plus exécutées et le programme continue son exécution.
- Pour éviter que la boucle, et donc le programme, ne s'exécute sans fin, le programmeur doit faire évoluer l'expression à chaque exécution de la boucle.

Exemple

Afficher des entiers consécutifs en utilisant une boucle `do { instructions; } while (expression);`

```

#include <iostream>
using namespace std;

int main(void)
{
    cout << "=== Afficher des entiers cons\202cutifs ===" << endl;

    int nPremier = 0;
    cout << "Entrez le premier entier : "; cin >> nPremier;

    int nDernier = 0;
    do
    {
        cout << "Entrez le dernier entier (≥" << nPremier << " : "; cin >>
nDernier;
    } while (nDernier<nPremier);

    int nN = nPremier;
    cout << "\tListe des entiers : ";

    do
    {
        cout << nN << " ";
        ++nN;
    } while (nN <= nDernier);
}

```

Boucle `for (initialisation; expression; itération) { instructions; }`

`for (initialisation; expression; itération) { instructions; }` est le troisième type de boucle disponible en C++.

Syntaxe

```

for (initialisation; expression; itération)
    instruction;

```

ou

```

for (initialisation; expression; iteration)
{
    instruction1;
    instruction2;
    ...
}

```

Sémantique

L'exécution de l'instruction `for (initialisation; expression; itération) { instructions; }` se décompose en deux étapes :

1. L'initialisation est réalisée une fois au début de l'instruction, préalablement à l'exécution de la boucle,
2. L'expression est testée :
 - Si elle est vraie, les instructions entre les accolades du bloc (ou l'instruction unique s'il n'y a pas de bloc) sont exécutées, puis l'itération, dans cet ordre, et l'étape n°2 recommence.
 - Si elle est fausse, l'instruction `for` est terminée, et l'exécution du programme se poursuit par l'instruction qui suit le `for`.

Exemple 1

```
nN = 4;
for (int i = 0; i < nN; ++i)
{
    cout << "i = " << i << "." << endl;
}
```

Remarques

- L'expression doit être de type entier (booléen), c'est-à-dire valoir la valeur 0 (« faux ») ou toute valeur différente de 0 (considérée alors comme « vrai »).
- La boucle `for (initialisation; expression; itération) { instructions; }` est très utilisée quand le nombre d'itérations de la boucle est connu *a priori*.
- En C++, la partie `initialisation` peut inclure la déclaration d'une variable (ce n'est pas le cas en C). Dans ce cas, la portée de cette variable est limitée à l'instruction `for`.
- La boucle `for (initialisation; expression; itération) { instructions; }` n'est pas exécutée si l'initialisation choisie rend l'expression fausse.
- Les parties `initialisation` et `itération` peuvent regrouper plusieurs instructions séparées par des virgules. Cette utilisation est pourtant à éviter car elle nuit à la clarté et à la compréhension du code.

Exemple : `for (int i = 0, int j = nN ; i < nN ; ++i, --j) { instructions; }.`

- Chaque partie du `for` peut être omise : `initialisation`, `expression`, `itération`.
Exemple de boucle sans fin : `for ( ; ; ) { instructions; }`, éventuellement arrêtée par l'utilisation d'une instruction `break`; Attention, ce n'est pas une bonne façon de programmer.

Exemple 2

```
#include <iostream>
using namespace std;

int main(void)
{
    cout << "=== Afficher des entiers cons\202cutifs ===" << endl;

    int nPremier = 0;
    cout << "Entrez le premier entier : "; cin >> nPremier;

    int nDernier = 0;
    do
    {
        cout << "Entrez le dernier entier (≥" << nPremier << " : "; cin >>
nDernier;
    } while (nDernier < nPremier);

    cout << "\tListe des entiers : ";

    for (int nN = nPremier; nN <= nDernier; ++nN)
    {
        cout << nN << " ";
    }
}
```

Choix des structures itératives

Le tableau suivant présente les critères qui peuvent être utilisés pour choisir la structure itérative la mieux adaptée à un contexte donné.

Nombre d'itérations	Instructions de la boucle	Type de boucle
Impossible à déterminer <i>a priori</i>	Exécution au moins 1 fois	do {...} while (expression)
	Pas d'exécution systématique minimum	do {...} while (expression)
Connu par le programmeur		for (initialisation, expression, itération) { instructions; }

Tableau 7 choix des structures itératives.:

Instructions de branchement inconditionnel

Le C++ comme le C proposent trois instructions de branchement inconditionnel : `break`, `continue` et `goto`. Ce paragraphe les présente car elles peuvent être rencontrées. Néanmoins, leur utilisation est à proscrire car elles rendent les programmes difficiles à comprendre et à maintenir. Les programmeurs de haut niveau se limitent uniquement à l'utilisation du `break` dans les instructions `switch (...) {...}`.

Instruction `break`

L'instruction `break` permet de quitter une instruction en cours (`while (...) {...}`, `do {...} while (...)`, `for (...) {...}` et `switch (...) {...}`). L'instruction qui est exécutée après un `break` est la première instruction qui suit l'instruction dont le programmeur souhaite arrêter l'exécution.

Syntaxe

```
break;
```

Remarque

- L'instruction `break;` est très utilisée dans les différents "case" des instructions `switch`.

Exemple

Lorsque le caractère nul `'\0'` est rencontré dans la variable `cChaine`, l'exécution se poursuit sur l'instruction qui suit la boucle `for`.

```
const int N = 32;
unsigned char pCh [N] = "Polytech";
int nLength = 0;

for (int i = 0; i < N; ++i)
{
    if (pCh[i] == '\0') break;
    nLength++;
}
cout << "La chaîne a " << nLength << "caractères." << endl;
```

Instruction `continue`

L'instruction `continue` impose le passage immédiat à l'itération suivante d'une boucle (`while (...) {...}`, `do {...} while (...)` ou `for (...) {...}`).

Syntaxe

```
continue;
```

Exemple

```
const int N = 32;
for (int i = 0; i < N; ++i)
{
    int k = i - 1;
    if (k == 0) continue;
    double dA = 1.0 / k;
}
```

Lorsque `k` est égal à zéro, on passe directement à l'itération suivante de la boucle (`i` est incrémenté et l'instruction `k = i - 1;` est exécutée). Tout se passe comme si les instructions de la boucle qui suivent l'exécution de l'instruction `continue` n'étaient pas exécutées dans cette itération.

Instruction `goto`

L'instruction `goto` permet un saut inconditionnel vers l'instruction qui suit l'étiquette spécifiée avec le `goto`.

Syntaxe

```
goto label;
...
label : instructions;
```

Remarques

-
- L'utilisation de l'instruction `goto` dans un programme C++ conduit immédiatement à un zéro au module.

Exemple

```
const int N = 32;
for (int j = 0; j < N; ++j)
{
    if (j == N/2) goto etiquette;
}

etiquette: cout << "Traitement spécifique cas " << N/2 << endl;
```